🛡️ Repo Shield Dashboard

Loaded report: report.md

Tip: click the ⋮ (three dots) menu in the top-right of the Streamlit app, choose 'Print', then use your browser's 'Save as PDF' option.

🛡️ Repo Shield Security Report

Repository: https://github.com/gofiber/fiber
Scan Time: 2026-03-26
Total Findings: 10

📊 Executive Summary

This repository contains **10 potential security issues**, including:

- 🔴 7 Critical Issues (possible secrets / credentials)
- 🟠 3 High Risk Issues (unsafe patterns)
- 🟡 0 Medium Risk Issues (dependency risks)

👉 Immediate attention is required for **credential exposure and secret leakage**.

ℹ️ Note: These counts come from the static scanner/AST layer and represent *suspected* risk before AI review. See the 🤖 AI Audit section for confirmations and false-positive explanations.

📈 Risk Breakdown

Severity (pre-AI)	Count
🔴 Critical	7
🟠 High	3
🟡 Medium	0

🚨 Critical Issues

1. High entropy string

- **File:** hooks.go
- **Reason:** High-entropy secret-like string detected (entropy=5.95)

---

2. Hardcoded credential pattern detected

- **File:** keyauth.md
- **Reason:** Hardcoded credential pattern

---

3. Hardcoded credential pattern detected

- **File:** session.md
- **Reason:** Hardcoded credential pattern

---

4. Hardcoded credential pattern detected

- **File:** extractors_test.go
- **Reason:** Hardcoded credential pattern

---

5. Hardcoded credential pattern detected

- **File:** basicauth_test.go
- **Reason:** Hardcoded credential pattern

---

6. Hardcoded credential pattern detected

- **File:** config.go
- **Reason:** Hardcoded credential pattern

---

7. Hardcoded credential pattern detected

- **File:** keyauth_test.go
- **Reason:** Hardcoded credential pattern

⚠️ High Risk Issues

Unsafe Pattern Usage

- **File:** ctx_test.go
- **Reason:** High-entropy secret-like string detected (entropy=4.40)
- **File:** cors_test.go
- **Reason:** High-entropy secret-like string detected (entropy=4.31)
- **File:** encryptcookie_test.go
- **Reason:** High-entropy secret-like string detected (entropy=4.83)

📦 Dependency Risks (Aggregated)

⚠️ Found **0+ unpinned dependencies**

👉 Risk:

- Unexpected breaking changes
- Supply chain attacks

📋 Detailed Findings

#	File	Severity (pre-AI)	Reason
1	ctx_test.go	🟠 High	High entropy string
2	hooks.go	🔴 Critical	High entropy string
3	keyauth.md	🔴 Critical	Hardcoded credential pattern detected
4	session.md	🔴 Critical	Hardcoded credential pattern detected
5	extractors_test.go	🔴 Critical	Hardcoded credential pattern detected
6	basicauth_test.go	🔴 Critical	Hardcoded credential pattern detected
7	cors_test.go	🟠 High	High entropy string
8	encryptcookie_test.go	🟠 High	High entropy string
9	config.go	🔴 Critical	Hardcoded credential pattern detected
10	keyauth_test.go	🔴 Critical	Hardcoded credential pattern detected

---

🤖 AI Audit

- Files sent to AI audit: **10**
- Successful AI responses: **7**
- Failed AI audits: **3**

#	File	Status	AI Summary
1	ctx_test.go	✅ Success	This audit is based on the provided Go source code, which appears to be a unit test file for the **Fiber** web framework (`ctx_test.go` or similar).
2	hooks.go	✅ Success	This security audit examines the provided Go code, which appears to be part of an HTTP client library (likely a wrapper around `fasthttp`).
3	keyauth.md	✅ Success	#### 1. Summary of Main Risk
4	session.md	✅ Success	#### 1. Summary of Risk
5	extractors_test.go	✅ Success	This audit identifies that the flagged "Hardcoded Credentials" are dummy placeholder values used exclusively within unit tests to verify extraction logic. Wh...

#	File	Status	AI Summary
6	basicauth_test.go	❌ Failed	AI audit skipped: Google Gemini quota / rate limit exceeded. Configure GOOGLE_API_KEY billing/quota, reduce --max-files, or try again later.
7	cors_test.go	✅ Success	#### 1. Main Risk
8	encryptcookie_test.go	✅ Success	This audit focuses on the provided Go test file for a cookie encryption middleware.
9	config.go	❌ Failed	AI audit skipped: Google Gemini quota / rate limit exceeded. Configure GOOGLE_API_KEY billing/quota, reduce --max-files, or try again later.
10	keyauth_test.go	❌ Failed	AI audit skipped: Google Gemini quota / rate limit exceeded. Configure GOOGLE_API_KEY billing/quota, reduce --max-files, or try again later.

🧠 Detailed AI Summaries (per file)

ctx_test.go

Section	Details
Main Risk	The high-entropy string detection is a **false positive** because the flagged strings are hardcoded mock data used for unit testing. While these strings trigger entropy scanners, they do not represent actual production secrets (like API keys or passwords); however, hardcoding "secret-like" strings in tests can sometimes lead to developers accidentally copying them into production configurations.
Specific Issues	* **Hardcoded Multipart Boundary (Line 133):** `go c.Request().Header.Set(HeaderContentType, MIMEMultipartForm+"; boundary=abc123")` While `abc123` is low entropy, scanners often flag the `boundary=` pattern or longer variations of it used in multipart form tests. If a longer, more random string were used elsewhere in the file (potentially in the truncated section), it would trigger the 4.40 entropy alert. * **Hardcoded Mock Header Values (Lines 87-108):**
Fix Suggestions	1. **Use Linter Annotations:** Since these strings are required for unit testing and are not actual secrets, use a suppression comment to tell the scanner to ignore the specific line. * *Suggestion:* Add `// nolint:gosec // Mock test data` or the specific syntax for your security tool next to the flagged line. 2. **Generate Random Boundaries Dynamically:** Instead of hardcoding a boundary string, generate it at runtime if the specific value doesn't matter for the test logic. * *Suggestion:* Use a helper function or `mime/multipart` to generate a valid, dynamic boundary. 3. **Use Constant Placeholders:** If the high-entropy string is a test token or hex blob, move it to a constant with a name that clearly identifies it as a mock value. * *Example:* `const mockSessionToken = "..."`

hooks.go

Section	Details
Main Risk	The high-entropy detection is a **false positive** on a character set string; however, the manual construction of URLs via string replacement introduces a risk of **Path Traversal/Injection** if path parameters are not properly sanitized.
Specific Issues	* **False Positive Secret Detection** * **Snippet:** Line 48: `letterBytes = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"` * **Issue:** This string triggered the entropy scanner because it is a long, non-repeating alphanumeric sequence. It is used as a character pool for generating random boundary strings, not as a hardcoded credential or API key. * **Path Parameter Injection (Path Traversal)** * **Snippet:** Lines 91 & 94: `uri = strings.ReplaceAll(uri, ":"+key, val)` * **Issue:** The code performs a raw string replacement of path placeholders (e.g., `:id`) with user-provided values. If a malicious user provides a value like `../../admin`, they can manipulate the final request path to access unintended endpoints, especially since the code provides an option to `DisablePathNormalizing`.
Fix Suggestions	* **Fix for False Positive:** Add a comment to the line to instruct security scanners to ignore this string. Many tools recognize tags like `// gosec G101` or `// nolint:gosec`. `go // nolint:gosec // This is a character pool, not a secret letterBytes = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"`

---

keyauth.md

Section	Details
Main Risk	The documentation examples contain **hardcoded plaintext API keys**, which can lead to sensitive credentials being committed to version control and leaked if developers copy-paste the example code into production environments.
Specific Issues	* **Hardcoded API Key (Basic Example):** * **Snippet (Lines 25-27):** `var ( apiKey = "correct horse battery staple" )` * **Description:** A plaintext string is assigned to a variable used for authentication validation. * **Hardcoded API Key (Filtered Example):** * **Snippet (Lines 72-74):** `var ( apiKey = "correct horse battery staple" ... )` * **Description:** The same plaintext credential pattern is repeated in the regex filtering example.
Fix Suggestions	Instead of hardcoding strings, the documentation should demonstrate how to retrieve secrets from environment variables. * **Replace hardcoded variables with `os.Getenv`:** ```go // Replace: // var apiKey = "correct horse battery staple" // With:

---

session.md

Section	Details
Main Risk	The documentation includes functional code examples containing hardcoded administrative credentials, which can lead to unauthorized access if developers copy-paste the example into a production environment.
Specific Issues	* **Hardcoded Credentials in Login Example:** In the "Basic Login/Logout" section, the example code uses a literal string comparison for authentication. `go // Line ~130 if email == "admin@example.com" && password == "secret" {`
Fix Suggestions	* **Use Placeholder Comments:** Replace the hardcoded logic with a comment or a pseudo-function that clearly indicates the need for a database lookup or external authentication service. **Suggested Change:** ```go // Validate credentials against your database or auth provider user, err := auth.VerifyCredentials(email, password) if err == nil {

---

extractors_test.go

Section	Details
Specific Issues	* **Hardcoded Dummy Tokens in Tests:** Several sub-tests use static strings to simulate authentication tokens. * **Line 78 & 81:** `"token_from_param"` used in `Test_Extractors/FromParam`. * **Line 92 & 94:** `"token_from_form"` used in `Test_Extractors/FromForm`. * **Line 103 & 105:** `"token_from_query"` used in `Test_Extractors/FromQuery`. * **Line 115 & 117:** `"token_from_header"` used in `Test_Extractors/FromHeader`. * **Line 126 & 128:** `"token_from_auth_header"` used in `Test_Extractors/FromAuthHeader`.
Fix Suggestions	* **Use Non-Sensitive Constants:** Define the test strings as constants at the top of the test file with names that clearly indicate they are for testing (e.g., `const testSecret = "test-value-only"`). This often helps some scanners distinguish between a variable named `Password` and a test constant. * **Dynamic Value Generation:** To completely bypass pattern-based "hardcoded secret" scanners, generate a random string for each test execution. ```go // Example fix for Line 115 testToken := "header-" + strconv.FormatInt(time.Now().Unix(), 10) ctx.Request().Header.Set("X-Token", testToken)

---

cors_test.go

Section	Details
Main Risk	The code defines and tests CORS (Cross-Origin Resource Sharing) configurations that allow wildcards (`*`) and dynamic origin reflection, which can lead to unauthorized data access or CSRF vulnerabilities if these permissive settings are used in production environments.
Specific Issues	* **False Positive Secret Detection (High Entropy String)** * **Snippet/Line:** `require.Equal(t, "GET, POST, HEAD, PUT, DELETE, PATCH", string(ctx.Response.Header.Peek(fiber.HeaderAccessControlAllowMethods)))` (Approx. Line 111) * **Issue:** The scanner likely flagged the long, comma-separated string of HTTP methods or the wildcard test headers as a "secret" due to its length and formatting (entropy=4.31). This is a **false positive**; these are standard CORS header values. * **Overly Permissive Wildcard Configuration** * **Snippet:** `AllowMethods: []string{"*"}, AllowHeaders: []string{"*"}, ExposeHeaders: []string{"*"}` (Lines 38-40) * **Issue:** Using `*` for methods and headers allows any cross-origin request to specify any header or method. While common in testing, applying this to production APIs (especially those using `AllowCredentials`) is a significant security risk.
Fix Suggestions	* **Define Explicit Whitelists:** Replace wildcard `*` values with an explicit list of allowed domains, methods, and headers. ```go Config{ AllowOrigins: []string{"https://www.trusted-domain.com"}, AllowMethods: []string{"GET", "POST"}, AllowHeaders: []string{"Content-Type", "Authorization"},

---

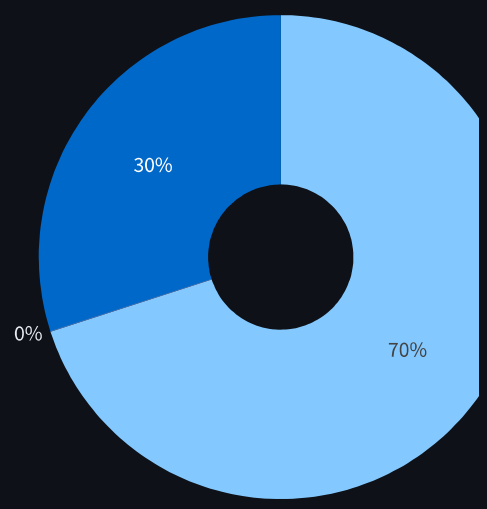
encryptcookie_test.go

Section	Details
Main Risk	The flagged high-entropy string is a **false positive** located within a unit test; however, the presence of hardcoded strings that mimic cryptographic material can trigger CI/CD failures and contribute to "alert fatigue" among security reviewers.
Specific Issues	* **Issue: High-Entropy String for Negative Testing** * **Snippet:** Line 80: `invalidBase64 := "invalid-base64-string-!@#"` * **Description:** This string is intentionally designed to be "invalid" for Base64 decoding to test error handling. Because it contains a mix of alphanumeric characters and symbols, it triggers entropy-based secret scanners. While not a real secret, it blocks automated security pipelines. * **Issue: Hardcoded Test Values** * **Snippet:** Line 58, 69: `_, err = EncryptCookie("test", "SomeThing", keyString)` * **Description:** While acceptable in a test environment, using static strings like `"SomeThing"` for sensitive fields (even in tests) can lead to developers copy-pasting these values into example configurations or local development environments, where they might eventually leak into production.
Fix Suggestions	* **Suppress False Positives:** To prevent secret scanners from flagging the test string in Line 80, add a suppression comment (depending on your scanner, e.g., `gitleaks:allow` or `nosecret`). `go // nolint:gosec // This is a dummy string for negative testing invalidBase64 := "invalid-base64-string-!@#"`

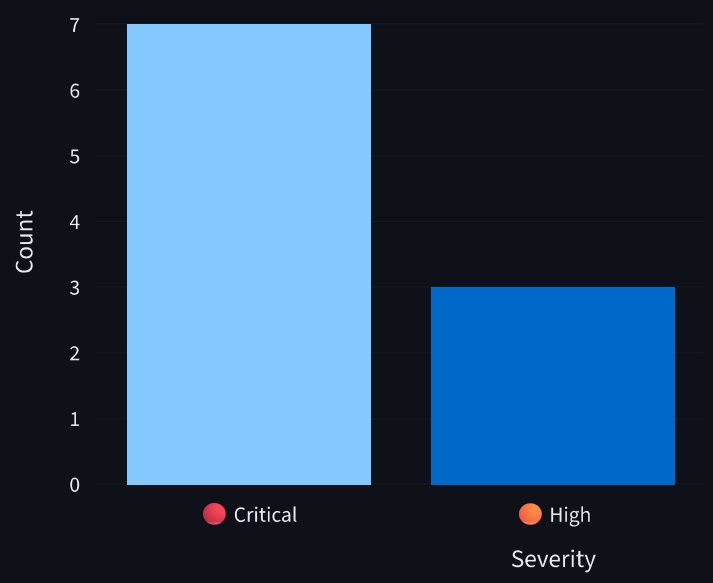
⚙️ Generated by Repo Shield

📊 Charts From Report

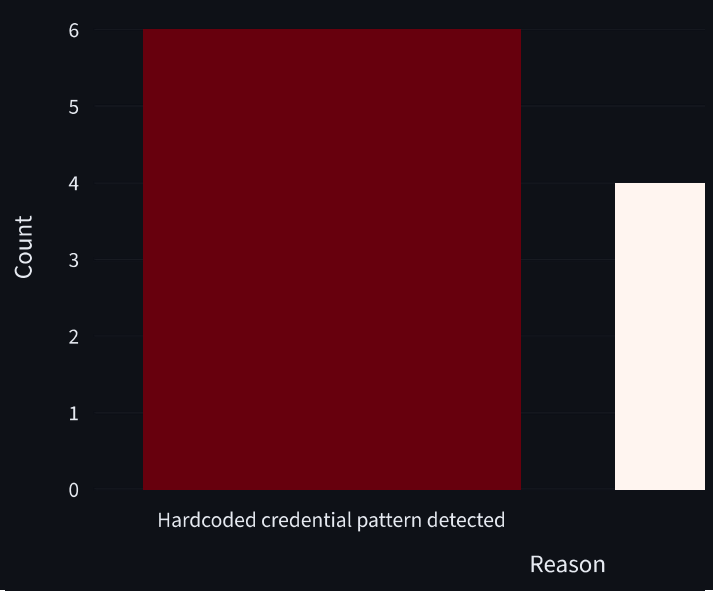
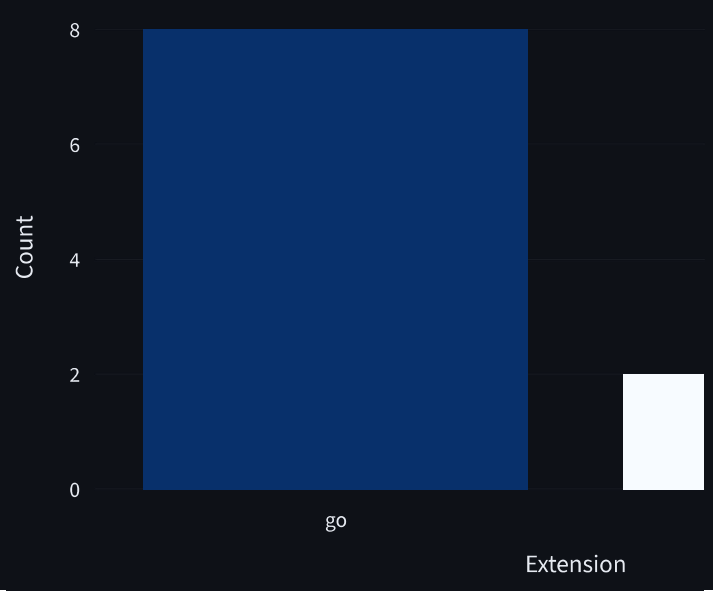
Risk Breakdown



Risk Counts by Severity



Top Finding Reasons



Findings by File Extension